Caché distribuido con Redis: patrones cache-aside y write-through, invalidación y consistencia eventual bajo alta concurrencia

Derrek Ureña Solis, José Arrieta Sancho
Escuela de Informática, Universidad Nacional de Costa Rica
jose.arrieta.sancho@est.una.ac.cr, derrek.urena.solis@est.una.ac.cr

***Resumen*—Los sistemas web empresariales usan cachés distribuidos para reducir la latencia y proteger las bases de datos, pero introducen una segunda copia del dato que puede quedar desactualizada. Este artículo analiza, con Redis como caso de estudio, los patrones cache-aside y write-through, la invalidación de claves, el uso de TTL y el problema de cache stampede bajo alta concurrencia. La comparación usa seis criterios explícitos (rendimiento, complejidad, costo, mantenibilidad, seguridad y madurez del ecosistema) y se apoya en literatura académica, documentación técnica primaria y reportes de ingeniería de empresas que operan cachés a gran escala. Los resultados indican que cache-aside con invalidación en la escritura y TTL con variación aleatoria (jitter) ofrece el mejor equilibrio para cargas dominadas por lectura que toleran consistencia eventual, mientras que write-through solo se justifica cuando un dato recién escrito debe leerse de inmediato. Ningún patrón evita por sí solo el stampede: se requiere combinar jitter, coalescencia de solicitudes y expiración probabilística. Se propone un marco de decisión por tipo de dato, su aplicación al sistema del equipo, y se discuten los límites del análisis y el trabajo futuro.**

***Palabras clave*—Redis, cache-aside, write-through, TTL, cache stampede.**

I. INTRODUCCIÓN

En las aplicaciones web empresariales actuales (plataformas de matrícula, comercio electrónico, banca en línea, servicios internos), miles de usuarios consultan los mismos datos al mismo tiempo y los picos de tráfico son parte normal de la operación. En ese contexto, la base de datos suele ser el primer cuello de botella. Un caché distribuido como Redis guarda en memoria los datos más consultados y evita ir a la base de datos en cada petición. El beneficio es real, pero el caché crea un problema nuevo: existen **dos copias del dato**, y el dato en caché crece inconsistente con su fuente con el paso del tiempo [1]. Decidir cuándo se actualiza o se borra esa copia determina si el sistema devuelve datos obsoletos, cuánta latencia agrega y si resiste picos de tráfico.

La literatura de sistemas distribuidos advierte que las decisiones de diseño de un caché son compromisos y no soluciones universales [2], y el caché a nivel de aplicación se adopta para cumplir requisitos de rendimiento cada vez más exigentes [3]. El tema importa hoy porque las arquitecturas modernas combinan varias bases de datos, servicios en la nube y tráfico variable, y porque un caché mal diseñado puede provocar caídas en cadena y no solo lentitud [4]. Aun así, en proyectos académicos y empresariales es común elegir un patrón por costumbre, sin comparar sus consecuencias.

Objetivo. Este trabajo analiza críticamente dos patrones, *cache-aside* y *write-through*, junto con los mecanismos que condicionan su comportamiento bajo concurrencia: la invalidación, la expiración por TTL (*time to live*) y el *cache stampede* (o *thundering herd*). El propósito es llegar a una posición técnica argumentada y aplicarla al sistema que el equipo construye en los laboratorios. Quedan fuera del alcance Redis como base de datos primaria y sus funciones de mensajería (pub/sub y colas).

El resto del artículo se organiza así: la Sección II presenta el marco conceptual; la Sección III compara los patrones con seis criterios; la Sección IV aplica el análisis al dominio del equipo; y la Sección V resume hallazgos, límites y trabajo futuro.

II. MARCO CONCEPTUAL

A. Consistencia eventual

Cuando los datos se replican, no se puede garantizar a la vez consistencia fuerte, disponibilidad y tolerancia a particiones de red [5]. Por eso muchos sistemas adoptan **consistencia eventual**: si dejan de llegar escrituras, todas las copias terminan convergiendo al mismo valor [6]. Un caché con TTL es un ejemplo típico: acepta que el dato esté desactualizado durante una ventana limitada a cambio de velocidad [2].

B. Redis como caché distribuido

Redis guarda los datos en memoria y permite asignar un TTL a cada clave [7]. Cuando la memoria se agota, aplica una política de expulsión configurable, por ejemplo LRU (menos usada recientemente) o LFU (menos frecuente) [7]. Para escalar, Redis Cluster reparte las claves entre nodos mediante 16384 *hash slots* [8]. Dos propiedades importan para este análisis: la replicación es **asíncrona** por defecto, así que una escritura confirmada puede perderse si el nodo primario falla antes de copiarla [9], y Redis Cluster no ofrece consistencia fuerte [8]. En consecuencia, el caché debe tratarse como una copia prescindible y no como fuente de verdad.

C. Cache-aside

La aplicación controla el caché. Al leer, consulta primero Redis; si no encuentra la clave (*miss*), lee la base de datos y guarda el resultado con un TTL. Al escribir, actualiza la base de datos e **invalida** (borra) la clave del caché [10], [11]. Solo se cachea lo que realmente se pide.

D. Write-through

Cada escritura pasa por el caché, que la propaga de forma síncrona a la base de datos antes de confirmar al cliente [11]. El caché queda alineado con la base de datos, a costa de más latencia de escritura y de guardar datos que quizá nunca se lean. Según las guías de arquitectura revisadas, este patrón se implementa en la capa de aplicación o con herramientas adicionales, y no como una operación propia de Redis [11].

E. Invalidación y condiciones de carrera

Invalidar significa borrar o marcar como inválida una copia cuando cambia el dato original. Como los cachés dinámicos se modifican tanto al leer (llenado tras un miss) como al escribir (invalidación), aparecen muchas condiciones de carrera [12]. El caso clásico en cache-aside es este: un lector obtiene el valor viejo de la base, un escritor actualiza la base y borra la clave, y después el lector guarda el valor viejo en el caché, donde permanece hasta que venza el TTL (Fig. 1) [10], [13]. Facebook mitigó el problema con *leases*: el caché entrega un token al primer cliente que falla y rechaza escrituras con tokens ya invalidados [13].



Fig. 1. Carrera lector/escritor en cache-aside que deja un valor viejo en el caché.

F. Cache stampede y fallos metaestables

El *cache stampede* ocurre cuando una clave muy solicitada expira y muchas peticiones concurrentes encuentran el miss al mismo tiempo; todas recalculan el valor y golpean la base de datos a la vez [13], [14]. Es un riesgo de fiabilidad, no solo de rendimiento. Un estudio de 22 fallos en 11 organizaciones muestra que los *fallos metaestables* (sobrecargas que persisten aun después de quitar el disparador, por ejemplo por reintentos) son frecuentes, y que un fallo de caché puede abrumar a la base de datos [4]. El stampede es una de las puertas de entrada a ese tipo de fallo.

G. Estado del arte reciente

Los trabajos recientes se concentran en medir y mejorar la consistencia de los cachés en producción. Meta reporta que el primer paso para mejorar la invalidación fue medir la consistencia de forma continua [12]. En la academia, An y Cao proponen políticas de caché que evitan que un cliente vea versiones más viejas que las que ya leyó, con mejoras de rendimiento sobre Redis [15]. Trabajos de 2024 y 2025 apuntan en la misma dirección: Mao et al. muestran que los TTL, la técnica dominante para acotar la obsolescencia, se vuelven muy costosos cuando se exige frescura de segundos, porque las lecturas a la base crecen en proporción inversa al TTL, y proponen reaccionar a las escrituras con invalidaciones o actualizaciones [16]; y Repin y Sidorov comparan cachés usados en producción y señalan que Redis Cluster ofrece solo consistencia eventual, de modo que las garantías más fuertes exigen un diseño basado en consenso, a costa de más latencia de escritura [17]. Por su parte, el estudio de fallos metaestables ilustra cómo una falla del caché puede abrumar a la base de datos [4]. Estas líneas indican que el problema no lo resuelve un patrón único, sino la combinación de técnicas y su medición, lo que motiva los criterios de la Sección III.

III. ANÁLISIS COMPARATIVO

A. Criterios y método

Se revisaron artículos de conferencias y revistas, documentación técnica primaria de Redis y guías de arquitectura de proveedores cloud. Los patrones se comparan con seis criterios definidos de antemano: **rendimiento** (latencia de lectura y escritura, y carga sobre la base de datos), **complejidad** de implementación, **costo** (memoria y operación), **mantenibilidad** (capacidad de entender, probar y observar el sistema), **seguridad** y **madurez del ecosistema** (documentación, soporte en herramientas y uso comprobado en producción). La Tabla I resume el resultado; los apartados siguientes lo justifican. Donde no hay medición publicada, se indica que la afirmación es un razonamiento propio de los autores.

TABLA I. COMPARACIÓN DE CACHE-ASIDE Y WRITE-THROUGH

Criterio	Cache-aside	Write-through
Rendimiento	Lecturas rápidas con hit; escritura barata (BD + borrado); el primer miss es costoso [10], [11]	Lecturas rápidas; escritura más lenta porque BD y caché son síncronos [11]
Complejidad	Lógica en la aplicación; carreras lector/escritor [12], [13]	Capa intermedia; doble escritura y recuperación ante fallos parciales [11]
Costo	Solo ocupa memoria lo que se lee [10]	Ocupa memoria con datos escritos que quizá nunca se lean [11]
Mantenibilidad	Código de caché disperso si no se encapsula; consistencia difícil de medir [12]	Un único punto de escritura, pero con más piezas que operar
Seguridad	Datos viejos de permisos hasta el TTL o la invalidación [18]	Menos ventana de permisos viejos; el caché igual duplica datos sensibles [18]
Madurez del ecosistema	Patrón documentado por Microsoft y AWS; uso a gran escala en Facebook [10], [11], [13]	Documentado por AWS; en Redis se construye en la aplicación o con herramientas adicionales [11]

Fuente: síntesis de las referencias indicadas en cada celda.

B. Rendimiento

Con un *hit*, ambos patrones leen desde memoria con la misma rapidez. La diferencia está en la escritura: cache-aside solo actualiza la base de datos y borra una clave, mientras que write-through suma dos pasos síncronos antes de responder

[11]. En cambio, cache-aside paga el costo en el primer miss tras una invalidación o expiración. La carga sobre la base se puede modelar de forma simple: $L_{BD} = \lambda_{lect} \cdot (1-h) + \lambda_{esc}$, donde h es la tasa de aciertos. Por eso un TTL demasiado corto baja h y sube la carga, y uno demasiado largo hace lo contrario con la frescura.

La evidencia sugiere que no basta con tener caché: la política con que se mantiene importa. An y Cao proponen políticas de caché que consideran consistencia y monotonicidad (no ver una versión más vieja de lo ya leído) y reportan hasta 39,09 % menos lecturas a la base de datos y 77,15 % más rendimiento en Redis sobre HBase, frente a políticas convencionales [15]. Además, Amazon señala que los patrones de petición desiguales generan *hot keys*, que son justamente las claves donde un stampede hace más daño [1].

C. Complejidad

Cache-aside parece sencillo, pero deja en la aplicación toda la responsabilidad de invalidar bien, y ahí viven las carreras descritas en la Sección II-E. Invalidar en lugar de actualizar el caché en la escritura es la práctica más segura, porque dos escrituras concurrentes que actualizan el caché pueden dejar el valor más antiguo como ganador [10], [13]. Write-through elimina parte de las carreras de lectura, pero introduce la **doble escritura**: si falla un paso a medias, caché y base quedan distintos, y hay que diseñar reintentos y reconciliación [11]. Ninguno elimina del todo la inconsistencia; solo cambian su forma.

D. Costo

El costo principal de un caché es la memoria, que es más cara que el disco, y se multiplica por cada réplica. Cache-aside carga datos bajo demanda, así que la memoria se llena con lo que se lee [10]. Write-through carga todo lo que se escribe; si los datos tienen acceso sesgado (pocas claves concentran las lecturas [19]), gran parte de esa memoria guarda datos que casi nadie lee. Este es un razonamiento de los autores, a validar midiendo la proporción de claves escritas que se leen después. En el otro lado del costo está la base de datos: un caché bien dimensionado permite una base más pequeña [1].

E. Mantenibilidad

En cache-aside, el código que consulta, llena e invalida tiende a repetirse por toda la aplicación. La mitigación es encapsularlo en una capa de acceso a datos (repositorio) que sea el único lugar donde se toca Redis. En write-through hay un solo punto de escritura, pero con más piezas que operar. En ambos casos, lo más difícil es **observar** la consistencia: Meta reporta que el primer paso para resolver la invalidación fue medirla, con un servicio que detecta entradas inconsistentes sin falsos positivos, y que su meta se expresa en más de diez nueves de consistencia [12]. Para un proyecto académico, el equivalente realista es registrar tasa de aciertos, latencia p95/p99 y carga en la base durante las expiraciones.

F. Seguridad

Redis está diseñado para ser accedido por clientes de confianza dentro de entornos de confianza. Su documentación recomienda no exponer el puerto, usar autenticación con listas de control de acceso (ACL) y cifrado TLS, que puede cubrir conexiones de clientes, enlaces de replicación y el bus del clúster [18]. Más allá de la configuración, el patrón elegido afecta la seguridad de los datos: en cache-aside, un cambio de permisos puede tardar en reflejarse hasta que se invalide la clave o venza el TTL, lo que significa que un usuario podría conservar acceso unos minutos de más. Además, las claves deben incluir el contexto del usuario o del inquilino (*tenant*) para no mezclar respuestas entre personas. El stampede también es un riesgo de disponibilidad: pedir muchas claves inexistentes puede saturar la base, y por eso conviene guardar el resultado "no existe" con TTL corto (*negative caching*).

G. Consistencia e invalidación en la práctica

Tres prácticas reducen la ventana de inconsistencia de cache-aside. La primera es **invalidar con borrado** en lugar de escribir el valor nuevo en el caché, como ya se explicó. La segunda es **versionar los valores**: guardar junto al dato un número de versión o una marca de tiempo y rechazar la escritura en el caché si su versión es menor a la que ya existe, de modo que un lector atrasado no pise un valor más nuevo. La tercera es el **segundo borrado diferido** (*delayed double delete*): repetir el borrado tras un breve intervalo, mayor que la duración típica de una lectura, para eliminar el valor viejo que un lector rezagado haya alcanzado a guardar. Los *leases* de Facebook generalizan esta idea con tokens [13]. Ninguna técnica ofrece consistencia fuerte; todas reducen la probabilidad de leer datos viejos.

Conviene además hablar de **obsolescencia acotada**: aceptar de forma explícita que un dato puede estar atrasado hasta un límite s (por ejemplo, el TTL) y diseñar el sistema alrededor de esa garantía. Esta noción aparece también en la literatura reciente sobre cachés, que usa una cota de obsolescencia para decidir qué copias son aún utilizables [15]. En write-through, el orden importa: si se escribe primero en la base y luego en el caché, un fallo intermedio deja un caché viejo, que se corrige invalidando o reintentando; si se escribe primero en el caché, un fallo posterior deja en el caché un dato que la base nunca recibió, lo cual es peor. Por eso, aun en write-through, la base de datos debe seguir siendo la fuente de verdad.

H. TTL y técnicas contra el stampede

El TTL es el límite máximo de obsolescencia que el negocio tolera. Si falla una invalidación, un dato nunca estará más viejo que su TTL, mientras que una invalidación perdida puede dejarlo atrasado indefinidamente [16]; por eso conviene definir TTL distintos por tipo de dato. El tamaño del stampede se puede estimar: si una clave caliente recibe λ peticiones por segundo y recalcularla toma δ segundos, hasta $\lambda \cdot \delta$ consultas idénticas llegan a la base al expirar (por ejemplo, 2000 peticiones/s y 0,2 s dan unas 400 consultas simultáneas).

La Tabla II resume las defensas. El *jitter* (variación aleatoria del TTL) evita que muchas claves cargadas al mismo tiempo expiren a la vez. La coalescencia deja que solo

una petición recalcule, por ejemplo con `SET clave valor NX PX tiempo`, pero los bloqueos distribuidos basados en Redis tienen límites ante fallos y pausas de proceso, por lo que sirven como optimización y no como garantía de corrección [20]. La expiración probabilística anticipada hace que cada lector decida, con probabilidad creciente, recalcular *antes* de que expire: Vattani et al. proponen recalcular si `ahora − δ·β·ln(rand()) ≥ expiración`, y demuestran que la estrategia es óptima sin coordinación entre clientes [14].

TABLA II. TÉCNICAS CONTRA EL CACHE STAMPEDE

Técnica	Qué evita	Costo o límite
TTL con jitter	Expiración simultánea de muchas claves	No protege una sola clave caliente
Coalescencia (single-flight)	Recálculos duplicados de la misma clave	Los demás esperan; el bloqueo distribuido no garantiza corrección [20]
Expiración probabilística	El miss mismo, al recalcular antes	Requiere ajustar β y δ; algunos recálculos extra [14]
Stale-while-revalidate	Latencia del miss	Sirve datos vencidos un tiempo
Precalentamiento	Arranque en frío tras despliegue	Hay que conocer las claves críticas

Fuente: elaboración propia a partir de [14], [20].

I. Resiliencia ante la caída o degradación del caché

Un caché no solo acelera: también se convierte en una dependencia de la que el sistema pasa a depender sin notarlo. Si Redis falla o se reinicia vacío, todas las lecturas caen a la vez sobre la base, lo que reproduce a gran escala el stampede; si además los clientes reintentan sin control, la sobrecarga puede sostenerse sola, que es el patrón de un fallo metaestable [4]. En pruebas con fallos simulados, un clúster Redis perdió datos al caer un nodo y quedó con dos líderes y datos inconsistentes al partirse la red [17]. Con cache-aside, la aplicación puede seguir sirviendo desde la base si la protege con límites de concurrencia, tiempos de espera cortos y reintentos con espera creciente y aleatoria. Con write-through, una caída de Redis afecta directamente a las escrituras, porque estas dependen del caché para completarse, por lo que se necesita una ruta alternativa que escriba solo en la base e invalide después. Probar el sistema con el caché apagado, como sugiere Amazon [1], es la forma más barata de descubrir estas dependencias antes de producción.

J. Madurez del ecosistema

Ambos patrones están descritos en guías de proveedores cloud [10], [11], pero con distinto respaldo. Cache-aside tiene más evidencia de uso a gran escala: Facebook lo emplea sobre Memcache, con borrado de la clave en la escritura y *leases* [13], y Meta sigue publicando sobre cómo medir su consistencia [12]. Write-through se documenta como estrategia [11], pero la evidencia pública revisada sobre su uso a escala con Redis es menor y exige construir la capa intermedia. Redis, en cambio, es un componente con documentación oficial sobre expulsión, clúster, replicación y seguridad [7], [8], [9], [18]. Un estudio de 2025 que compara cachés en producción describe a Redis Cluster como el más usado en sistemas industriales y, por tanto, el más probado en condiciones reales, y señala que Memcache incorpora un mecanismo de leases contra el thundering herd que Redis no trae integrado [17]. En el stack del equipo, el soporte de Spring para cachés con Redis facilita cache-aside (Sección IV-D).

K. Síntesis

Para cargas dominadas por lectura, cache-aside ofrece mejor rendimiento de escritura, menor costo de memoria y mejor resiliencia: si Redis cae, la aplicación puede seguir leyendo de la base, aunque con más carga, y Amazon recomienda probar el servicio con el caché desactivado para validar esas protecciones [1]. Write-through reduce la ventana de datos viejos, pero a cambio de latencia, complejidad y memoria. Su ventaja real aparece solo cuando se exige leer de inmediato lo recién escrito. Las técnicas anti-stampede se complementan: el jitter previene expiraciones masivas, la expiración probabilística las adelanta y la coalescencia limita el daño cuando ocurren.

IV. APLICACIÓN AL DOMINIO PROPIO

A. Descripción del sistema

LESCO CR es una plataforma de gestión de cursos de Lengua de Señas Costarricense (LESCO) para una institución educativa que imparte esa lengua. Administra el catálogo de cursos, los instructores asignados a cada lección, la matrícula de estudiantes con su cobro (SINPE Móvil, tarjeta o transferencia) y el material didáctico: videos, imágenes y recursos multimedia de señas, además de un diccionario digital de señas con comentarios de la comunidad.

El backend está hecho en Java con Spring Boot. Los datos estructurados (cursos, matrículas, pagos) viven en PostgreSQL y el contenido flexible (diccionario, comentarios, metadatos de multimedia) en MongoDB. Flyway versiona el esquema de la base de datos y Testcontainers permite ejecutar las pruebas en un entorno limpio y reproducible. Como hay dos bases, el caché funciona como una tercera copia del dato con dos posibles orígenes, lo que aumenta la importancia de definir con claridad cuál es la fuente de verdad de cada entidad.

B. Perfil de carga

La carga de LESCO CR está dominada por la lectura. Los estudiantes consultan el catálogo, el diccionario y el material didáctico con mucha más frecuencia de la que se crean o modifican cursos, de modo que la mayoría de las peticiones son lecturas. El tráfico no es uniforme: se concentra en la apertura de matrícula de cada periodo, cuando la mayoría de los usuarios consulta al mismo tiempo los tres cursos de mayor demanda (por ejemplo, LESCO I, LESCO II y Conversacional). Esas tres claves del catálogo se comportan como hotkeys, ya que pocas claves concentran la mayor parte de las lecturas [1], [19].

Como el sistema aún no está en producción, se plantea un escenario hipotético para dimensionar el riesgo. Si durante la apertura de matrícula cada curso caliente recibe

λ = 2000 peticiones/s y la consulta a PostgreSQL tarda δ = 0,2 s, al expirar la clave llegarían hasta $\lambda \cdot \delta \approx 400$ consultas idénticas y simultáneas a la base por cada curso. Si los tres cursos expiran a la vez, la cifra asciende a unas 1200. Fuera de esos picos la carga es baja y estable: el diccionario recibe lecturas constantes con pocas escrituras, y los comentarios tienen escrituras frecuentes pero un volumen reducido. Estos valores son supuestos de diseño, no mediciones, y se validan con la prueba de la Sección IV-E.

El cálculo de 1200 consultas para tres claves simultáneas justifica de paso el uso de jitter en la Tabla III, porque muestra qué pasa si las tres claves expiran juntas.

C. Decisión argumentada

Decisión: sí se aplica el enfoque analizado, de forma selectiva. Se propone **cache-aside como patrón por defecto** y una decisión distinta por tipo de dato (Tabla III); los cupos, los pagos y los datos de tarjeta quedan fuera del caché. Los valores de TTL son puntos de partida que deben calibrarse con las pruebas de la Sección IV-E.

TABLA III. DECISIÓN POR TIPO DE DATO EN LESCO CR

Dato (almacén)	Tolerancia al atraso	Decisión	TTL inicial
Catálogo de cursos (PostgreSQL)	Minutos	Cache-aside (sin cupos); invalidar tras el commit; single-flight y expiración probabilística en los 3 cursos	10–30 min + jitter
Instructores por lección (PostgreSQL)	Minutos	Cache-aside; invalidar al reasignar	5–15 min + jitter
Entradas del diccionario (MongoDB)	Horas	Cache-aside con TTL largo	1–6 h + jitter
Comentarios (MongoDB)	Baja	Cache-aside; invalidar al publicar o moderar	1–2 min
Permisos de acceso: matrícula vigente (PostgreSQL)	Muy baja	Cache-aside; invalidar al cambiar matrícula o pago	1–5 min
Cupos y estado de pago (PostgreSQL)	Casi nula	Sin caché; decide la transacción en la base	N/A
Datos de tarjeta o cobro	Ninguna	Nunca cachear	N/A

Fuente: elaboración propia a partir del análisis de la Sección III.

Las razones que sustentan esta decisión son las siguientes:

Cache-aside por defecto. Catálogo, instructores y diccionario son de lectura dominante y toleran minutos u horas de atraso. Cache-aside ocupa memoria solo con lo que se consulta [10], escribe más barato que write-through [11] y, si Redis falla, la plataforma puede seguir leyendo de las bases [1].

Write-through se descarta. Ningún dato del sistema exige lectura inmediata de lo recién escrito con la fuerza que justifique su costo de latencia, memoria y doble escritura [11]. La única necesidad cercana es que quien publica un comentario lo vea de inmediato (*read-your-writes*), que se resuelve con invalidación y mostrando el comentario en la vista de su autor, sin adoptar el patrón completo.

La base de datos decide cupos y pagos. El número de cupos y el estado de un pago no se sirven desde el caché. Hoy el cupo disponible se calcula como `cupoTotal` menos las matrículas activas, en `CursoService` y `MatriculaService`, y el código actual no usa `@Version` ni `@Lock`. Por eso, dos matrículas simultáneas podrían superar el cupo, un riesgo que existe con o sin caché. Como cambio propuesto, esa operación debe ejecutarse dentro de una transacción con control de concurrencia en PostgreSQL (bloqueo optimista con `@Version` o bloqueo pesimista con `@Lock`), y el cupo calculado no se almacena en Redis.

Multimedia fuera de Redis. Los videos e imágenes son archivos pesados y deben almacenarse en un servicio de objetos o una red de distribución de contenido. En Redis solo se guardarían metadatos o direcciones, con un TTL menor a la vigencia de cualquier dirección firmada.

Seguridad. Un cambio de matrícula, un reembolso o una moderación podría tardar en reflejarse hasta que se invalide la clave o venza el TTL, así que permisos y comentarios llevan TTL corto e invalidación explícita. Las claves incluyen el identificador del usuario cuando la respuesta es personal, y Redis se configura con ACL y TLS [18]. Los datos de tarjeta nunca se cachean.

Defensa anti-stampede por niveles. Jitter en todos los TTL; coalescencia en las claves del catálogo y de las entradas más buscadas; y expiración probabilística en los tres cursos durante la apertura de matrícula [14].

D. Implementación en el stack

La abstracción de caché de Spring implementa cache-aside casi de forma literal: `@Cacheable` para leer y `@CacheEvict` para invalidar, con `RedisCacheManager` como respaldo y un TTL configurable por caché. El jitter no viene incluido; se logra calculando el TTL con un componente aleatorio, y conviene verificar qué ofrece la versión de Spring Data Redis que usa el equipo. La opción `sync = true` de `@Cacheable` evita que varios hilos recalculen la misma clave, pero solo dentro de una misma instancia de la aplicación; con varias instancias se necesita un bloqueo en Redis, con los límites señalados en [20].

La invalidación debe ejecutarse **después de confirmar la transacción** (por ejemplo, con `@TransactionalEventListener` en fase `AFTER_COMMIT`). Si se borrara antes, un lector podría volver a llenar el caché con el valor viejo, que es la carrera de la Fig. 1 [12], [13]. Flyway aporta una ventaja adicional: cuando una migración cambia la forma de un dato, versionar las claves con un prefijo (por ejemplo, `v2:curso:LESCO-101`) hace que las entradas viejas se ignoren sin vaciar Redis.

E. Riesgos, medición y validación

Como no hay datos de producción, la decisión se valida con un experimento propio. Con Testcontainers se levanta un Redis real y se lanzan 400 hilos (la cifra $\lambda \cdot \delta$ de la Sección IV-B para un curso) que piden a la vez la misma clave recién expirada, contando cuántas veces se ejecuta la consulta al repositorio. La hipótesis es que, sin protección, el conteo se acerca al número de hilos, y con coalescencia baja a una sola consulta. Después, una prueba de carga con k6 sobre el entorno completo simula la apertura de matrícula y compara dos configuraciones: sin protección anti-stampede, y con jitter más coalescencia más expiración probabilística.

Se registran la tasa de aciertos, la latencia p95 y p99 del catálogo, el número de consultas a PostgreSQL y MongoDB durante la expiración, y las lecturas obsoletas observadas tras editar un curso. Los umbrales de aceptación se fijan antes de ejecutar el experimento: por ejemplo, tasa de aciertos de al menos 99 % en las claves del catálogo y no más de 1 consulta a la base por clave en cada expiración (frente a unas 400 en el escenario sin protección). El umbral de 99 % es conservador: con $\lambda = 2000$ peticiones/s y un TTL de 10 minutos, una sola recarga por ventana implica una tasa ideal de $1 - 1/(\lambda \cdot \text{TTL}) \approx 99{,}9999$ %, y el margen cubre arranques en frío y expulsiones. Si los resultados no mejoran con el caché, o la concurrencia real resulta baja, la conclusión válida podría ser no cachear ciertos datos, y eso también debe argumentarse con estas métricas.

F. Qué cambiaría, qué costaría y qué se ganaría

Qué cambiaría. La arquitectura incorporaría un servicio Redis (por ejemplo, un contenedor adicional en el entorno de desarrollo), la dependencia de Spring Data Redis, una configuración de cachés con su TTL, una capa que encapsule las lecturas y oyentes de eventos que invalidan después del commit. Flyway y Testcontainers se ajustan para versionar claves y levantar Redis en las pruebas.

Qué costaría. Memoria dedicada (unos 64 MB de reserva, estimados con estos supuestos: catálogo, instructores y permisos menos de 2 MB; entradas de diccionario no más de 10 MB; comentarios paginados; ≈ 15 MB; lo que suma como máximo 27 MB y un factor de 2 por metadatos y fragmentación de Redis da hasta ≈ 54 MB redondeados a 64 MB), un componente más que operar y monitorear, tiempo de desarrollo para la invalidación y las pruebas, y el riesgo permanente de servir datos atrasados dentro del TTL.

Qué se ganaría. Menos consultas a PostgreSQL y MongoDB en los picos de matrícula, menor latencia en catálogo y diccionario, y protección de las bases frente al stampede. Estas ganancias son hipótesis hasta medirlas (al menos 99 % menos consultas a PostgreSQL en las claves calientes durante el pico, porque pasan de unas 2000 consultas por segundo a una por ventana de TTL; y una latencia p95 de lectura del catálogo de 50 ms o menos, frente a los ≈ 200 ms de la consulta directa, valor δ de la Sección IV-B); si no se confirman, la decisión debe revisarse.

V. CONCLUSIONES

A. Hallazgos

1. Cache-aside con invalidación en escritura es el punto de partida razonable en sistemas dominados por lectura: es simple, barato en memoria y resiliente a la caída del caché, y su inconsistencia se acota con TTL e invalidación.
2. Write-through reduce la obsolescencia, pero cuesta latencia de escritura, memoria y complejidad por la doble escritura. Se justifica solo con requisitos explícitos de lectura inmediata.
3. Ningún patrón evita por sí solo el stampede. La combinación de TTL con jitter, coalescencia y expiración probabilística es la defensa mejor documentada.
4. Como Redis replica de forma asíncrona y no ofrece consistencia fuerte, el caché debe diseñarse como una copia prescindible y la base de datos como fuente de verdad.
5. Para LESCO CR se adopta cache-aside como patrón por defecto, con invalidación tras el commit, TTL diferenciado con jitter y defensa anti-stampede reforzada en las tres claves calientes del catálogo; cupos y pagos quedan fuera del caché. Esta decisión debe confirmarse con la prueba de la Sección IV-E.

B. Límites del análisis

El análisis es principalmente teórico y documental: las comparaciones de costo y mantenibilidad son razonamientos respaldados por fuentes, no mediciones propias. Gran parte de la evidencia proviene de empresas de escala muy superior a la de un proyecto académico, por lo que algunos problemas (consistencia en múltiples centros de datos) pueden no aparecer en el sistema del equipo. Además, LESCO CR aún no está en producción, por lo que los números de carga están pendientes y las decisiones de TTL son iniciales. Se excluyeron deliberadamente write-behind, Redis como base primaria y mensajería.

C. Trabajo futuro

Se propone, una vez disponible tráfico real, medir en el propio sistema el efecto del TTL, del jitter y de la expiración probabilística bajo una prueba de carga controlada; evaluar el caché del lado del cliente que ofrece Redis; y estudiar mecanismos de consistencia más fuertes, como los *leases* o las políticas con monotonicidad [13], [15], si los requisitos del negocio los exigen.

Este trabajo comparó cache-aside y write-through con seis criterios y concluye que no existe un patrón universal. La elección depende de cuánto atraso tolera cada tipo de dato. Para sistemas dominados por lectura como LESCO CR, cache-aside con invalidación tras el commit, TTL con jitter y defensas anti-stampede ofrece el mejor equilibrio, siempre que la base de datos siga siendo la fuente de verdad.

REFERENCIAS

[1] M. Brinkley and J. Chhabra, "Caching challenges and strategies," *Amazon Builders' Library*. [Online]. Available: https://aws.amazon.com/builders-library/caching-challenges-and-strategies/ (accessed Oct. 2026).

[2] M. Kleppmann and C. Riccomini, *Designing Data-Intensive Applications*, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2026.

[3] J. Mertz, I. Nunes, L. D. Toffola, M. Selakovic, and M. Pradel, "Satisfying increasing performance requirements with caching at the application level," *IEEE Softw.*, vol. 38, no. 3, pp. 87–95, 2021, doi: 10.1109/MS.2020.3033508.

[4] L. Huang *et al.*, "Metastable failures in the wild," in *Proc. 16th USENIX Symp. Operating Systems Design and Implementation (OSDI)*, 2022, pp. 73–90.

[5] S. Gilbert and N. Lynch, "Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services," *ACM SIGACT News*, vol. 33, no. 2, pp. 51–59, 2002, doi: 10.1145/564585.564601.

[6] W. Vogels, "Eventually consistent," *Commun. ACM*, vol. 52, no. 1, pp. 40–44, Jan. 2009, doi: 10.1145/1435417.1435432.

[7] Redis, "Key eviction," *Redis Documentation*. [Online]. Available: https://redis.io/docs/latest/develop/reference/eviction/ (accessed Oct. 2026).

[8] Redis, "Redis cluster specification," *Redis Documentation*. [Online]. Available: https://redis.io/docs/latest/operate/oss_and_stack/reference/cluster-spec/ (accessed Oct. 2026).

[9] Redis, "Redis replication," *Redis Documentation*. [Online]. Available: https://redis.io/docs/latest/operate/oss_and_stack/management/replication/ (accessed Oct. 2026).

[10] Microsoft, "Cache-Aside pattern," *Azure Architecture Center*. [Online]. Available: https://learn.microsoft.com/azure/architecture/patterns/cache-aside (accessed Oct. 2026).

[11] Amazon Web Services, "Database caching strategies using Redis," AWS Whitepaper. [Online]. Available: https://docs.aws.amazon.com/whitepapers/latest/database-caching-strategies-using-redis/ (accessed Oct. 2026).

[12] Meta Engineering, "Cache made consistent: Meta's cache invalidation solution," Jun. 8, 2022. [Online]. Available: https://engineering.fb.com/2022/06/08/core-infra/cache-made-consistent/

[13] R. Nishtala *et al.*, "Scaling Memcache at Facebook," in *Proc. 10th USENIX Symp. Networked Systems Design and Implementation (NSDI)*, 2013, pp. 385–398.

[14] A. Vattani, F. Chierichetti, and K. Lowenstein, "Optimal probabilistic cache stampede prevention," *Proc. VLDB Endow.*, vol. 8, no. 8, pp. 886–897, 2015, doi: 10.14778/2757807.2757813.

[15] S. An and Y. Cao, "Making cache monotonic and consistent," *Proc. VLDB Endow.*, vol. 16, no. 4, pp. 891–904, 2022, doi: 10.14778/3574245.3574271.

[16] Z. Mao, R. Iyer, S. Shenker, and I. Stoica, "Revisiting cache freshness for emerging real-time applications," in *Proc. 23rd ACM Workshop on Hot Topics in Networks (HotNets '24)*, Irvine, CA, USA, Nov. 2024, doi: 10.1145/3696348.3696858.

[17] V. Repin and A. Sidorov, "Distributed caching system with strong consistency model," *Front. Comput. Sci.*, vol. 7, Art. no. 1511161, May 2025, doi: 10.3389/fcomp.2025.1511161.

[18] Redis, "Redis security," *Redis Documentation*. [Online]. Available: https://redis.io/docs/latest/operate/oss_and_stack/management/security/ (accessed Oct. 2026).

[19] B. Atikoglu *et al.*, "Workload analysis of a large-scale key-value store," in *Proc. ACM SIGMETRICS*, 2012, pp. 53–64, doi: 10.1145/2254756.2254766.

[20] M. Kleppmann, "How to do distributed locking," Feb. 2016. [Online]. Available: https://martin.kleppmann.com/2016/02/08/how-to-do-distributed-locking.html